4.2 Visibility and Lifetime (Global Variables, etc.)

We haven't said so explicitly, but variables are channels of communication within a program. You set a variable to a value at one point in a program, and at another point (or points) you read the value out again. The two points may be in adjoining statements, or they may be in widely separated parts of the program.

How long does a variable last? How widely separated can the setting and fetching parts of the program be, and how long after a variable is set does it persist? Depending on the variable and how you're using it, you might want different answers to these questions.

The *visibility* of a variable determines how much of the rest of the program can access that variable. You can arrange that a variable is visible only within one part of one function, or in one function, or in one source file, or anywhere in the program. (We haven't really talked about source files yet; we'll be exploring them soon.)

Why would you want to limit the visibility of a variable? For maximum flexibility, wouldn't it be handy if all variables were potentially visible everywhere? As it happens, that arrangement would be *too* flexible: everywhere in the program, you would have to keep track of the names of all the variables declared anywhere else in the program, so that you didn't accidentally re-use one. Whenever a variable had the wrong value by mistake, you'd have to search the entire program for the bug, because any statement in the entire program could potentially have modified that variable. You would constantly be stepping all over yourself by using a common variable name like `i` in two parts of your program, and having one snippet of code accidentally overwrite the values being used by another part of the code. The communication would be sort of like an old party line--you'd always be accidentally interrupting other conversations, or having your conversations interrupted.

To avoid this confusion, we generally give variables the narrowest or smallest visibility they need. A variable declared within the braces `{}` of a function is visible only within that function; variables declared within functions are called *local variables*. If another function somewhere else declares a local variable with the same name, it's a different variable entirely, and the two don't clash with each other.

On the other hand, a variable declared outside of any function is a *global variable*, and it is potentially visible anywhere within the program. You use global variables when you *do* want the communications path to be able to travel to any part of the program. When you declare a global variable, you will usually give it a longer, more descriptive name (not something generic like `i`) so that whenever you use it you will remember that it's the same variable everywhere.

Another word for the visibility of variables is *scope*.

How long do variables last? By default, local variables (those declared within a function) have *automatic duration*: they spring into existence when the function is called, and they (and their values) disappear when the function returns. Global variables, on the other hand, have *static duration*: they last, and the values stored in them persist, for as long as the program does. (Of course, the values can in general still be overwritten, so they don't necessarily persist forever.)

Finally, it is possible to split a function up into several source files, for easier maintenance. When several source files are combined into one program (we'll be seeing how in the next chapter) the compiler must have a way of correlating the global variables which might be used to communicate between the several source files. Furthermore, if a global variable is going to be useful for communication, there must be exactly one of it: you wouldn't want one function in one source file to store a value in one global variable named `globalvar`, and then have another function in another

source file read from a *different* global variable named `globalvar`. Therefore, a global variable should have exactly one *defining instance*, in one place in one source file. If the same variable is to be used anywhere else (i.e. in some other source file or files), the variable is declared in those other file(s) with an *external declaration*, which is not a defining instance. The external declaration says, ``hey, compiler, here's the name and type of a global variable I'm going to use, but don't define it here, don't allocate space for it; it's one that's defined somewhere else, and I'm just referring to it here.'' If you accidentally have two distinct defining instances for a variable of the same name, the compiler (or the linker) will complain that it is ``multiply defined.''

It is also possible to have a variable which is global in the sense that it is declared outside of any function, but private to the one source file it's defined in. Such a variable is visible to the functions in that source file but not to any functions in any other source files, even if they try to issue a matching declaration.

You get any extra control you might need over visibility and lifetime, and you distinguish between defining instances and external declarations, by using *storage classes*. A storage class is an extra keyword at the beginning of a declaration which modifies the declaration in some way. Generally, the storage class (if any) is the first word in the declaration, preceding the type name. (Strictly speaking, this ordering has not traditionally been necessary, and you may see some code with the storage class, type name, and other parts of a declaration in an unusual order.)

We said that, by default, local variables had automatic duration. To give them static duration (so that, instead of coming and going as the function is called, they persist for as long as the function does), you precede their declaration with the `static` keyword:

```
static int i;
```

By default, a declaration of a global variable (especially if it specifies an initial value) is the defining instance. To make it an external declaration, of a variable which is defined somewhere else, you precede it with the keyword `extern`:

```
extern int j;
```

Finally, to arrange that a global variable is visible only within its containing source file, you precede it with the `static` keyword:

```
static int k;
```

Notice that the `static` keyword can do two different things: it adjusts the duration of a local variable from automatic to static, or it adjusts the visibility of a global variable from truly global to private-to-the-file.

To summarize, we've talked about two different attributes of a variable: visibility and duration. These are orthogonal, as shown in this table:

	duration:	
visibility:	automatic	static
local	normal local variables	`static` local variables
global	N/A	normal global variables

We can also distinguish between file-scope global variables and truly global variables, based on the presence or absence of the `static` keyword.

We can also distinguish between external declarations and defining instances of global variables, based on the presence or absence of the `extern` keyword.

Read sequentially: prev next up top